

Manual de compilación de microservicios

Proyecto: Sistema de Cambio de AFORE

Autor: Ricardo de Jesús Ruiz Marín

Fecha: octubre de 2025

ÍNDICE

1. Propósito del documento
2. Descripción general del sistema
3. Requisitos previos
4. Estructura del proyecto
5. Configuración de la base de datos MySQL
6. Procedimiento de compilación
7. Ejecución de los microservicios
8. Verificación en Postman
9. Verificación en Swagger
10. Observaciones finales

1. PROPÓSITO DEL DOCUMENTO

El presente manual tiene como objetivo describir de forma clara y ordenada el proceso de compilación, configuración y ejecución de los microservicios que conforman el sistema “Cambio de AFORE”, desarrollado en Java Spring Boot con base de datos MySQL.

Este documento permite a cualquier desarrollador replicar el entorno de ejecución de manera local y funcional.

2. DESCRIPCIÓN GENERAL DEL SISTEMA

El sistema está desarrollado bajo una arquitectura de microservicios, donde cada módulo cumple una función específica dentro del proceso de cambio de AFORE. Todos los microservicios se conectan a una misma base de datos central,

compartiendo tablas relacionadas para garantizar coherencia e integridad en la información. Se cuenta con un total de 5 microservicios, los cuales son:

- MS-Solicitud: Gestiona las solicitudes de cambio de AFORE realizadas por los usuarios.
- MS-Clientes: Administra la información personal y laboral de los clientes registrados.
- MS-Estado_Solicitud: Controla los estados del proceso (En proceso, Validada, Rechazada).
- MS-Correccion_De_Datos: Permite modificar datos de los clientes que realizaron solicitudes antes de su validación final.
- MS-Video_De_Consentimiento: Administra la carga y almacenamiento de los videos de consentimiento grabados por los clientes.

3. REQUISITOS PREVIOS

Software necesario:

- Java JDK 17
- Spring Tool Suite 4.32.0
- Apache Maven 3.8.9
- MySQL Workbench 8.0
- MySQL Server en ejecución (puerto 3306)
- Postman
- Git y GitHub
- Swagger UI

Configuración previa:

1. Verificar la variable de entorno JAVA_HOME.
2. Asegurarse de que Maven esté instalado (mvn -v).
3. Confirmar que MySQL Server esté activo y con acceso mediante un usuario válido.

4. ESTRUCTURA DEL PROYECTO

El repositorio tiene la siguiente organización:

MS/

- |— Clientes/
- |— Solicitud/
- |— Estado_Solicitud/
- |— Correccion_De_Datos/
- |— Video_De_Consentimiento/
- |— Documentacion/

Cada microservicio funciona de forma independiente y contiene su propio archivo pom.xml, además de que funcionan en diferentes puertos.

5. CONFIGURACIÓN DE LA BASE DE DATOS MYSQL

1. Abrir MySQL Workbench.
2. Crear una única base de datos central para todos los microservicios:
3. Ejecutar el script SQL incluido en el repositorio (DB_Script.sql) para generar las tablas necesarias.
4. Verificar en sts que cada microservicio tenga su archivo application.properties configurado para apuntar a la misma base de datos:

```
spring.application.name= nombre del proyecto (microservicio)

spring.datasource.url=jdbc:mysql://localhost:3306/afores
spring.datasource.username=root
spring.datasource.password= contraseña del usuario root para usar workbench

spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.dialect=org.hibernate.dialect.MySQL8Dialect
```

server.port=8080 (va a ir cambiando dependiendo del microservicio, cada uno debe tener un puerto diferente)

6. PROCEDIMIENTO DE COMPILACIÓN

Para compilar un microservicio tienes que seguir los siguientes pasos:

1. Dar click sobre el proyecto
2. Seleccionar debug as
3. Posteriormente, seleccionar la opción "4 Maven Build..."
4. En la ventana emergente, escribir *clean install package* en el campo goals y borrar el pom.xml del campo profiles
5. Una vez hecho esto, dar click en Apply y debug

7. EJECUCIÓN DE LOS MICROSERVICIOS

Cada microservicio se ejecuta de forma independiente pero todos utilizan la misma base de datos.

Para ejecutar los microservicios, puedes hacerlo desde STS con **Debug as** → **Spring Boot App**

Cada microservicio corre en diferentes puertos, los utilizados en este caso fueron:

Microservicio	Puerto
Clientes	8080
Estatus_Solicitud	8081
Solicitudes	8082
Correccion_De_Datos	8083
Video_De_Consentimiento	8084

8. VERIFICACIÓN EN POSTMAN

1. Abrir Postman (o la extensión Postman en Visual Studio Code)
2. Escribir la URL endpoint del microservicio a probar, por ejemplo, <http://localhost:8080/clientes>
3. Agregar las peticiones básicas (GET, POST, PUT, DELETE)
4. En el caso del método POST y PUT, escribir la información necesaria en body -> raw -> JSON
5. Para ejecutarlo, dé click en enviar

9. VERIFICACIÓN EN SWAGGER

Cada microservicio incluye Swagger para documentar y probar los endpoints disponibles.

Pasos para verificación:

1. Asegúrese de que el microservicio esté en ejecución.
2. Abra un navegador y acceda a:
3. <http://localhost:8080/swagger-ui/index.html>

NOTA: el puerto puede variar según la configuración del microservicio, por ejemplo 8081, 8082, etc.

4. Verifique que:
 - Los controladores y endpoints aparecen correctamente documentados.
 - Cada operación puede ejecutarse directamente desde la interfaz Swagger.
 - Las respuestas de los endpoints coinciden con lo esperado en la base de datos.
5. Ejemplo de URLs según microservicio:
 - Clientes → <http://localhost:8080/swagger-ui/index.html>
 - Solicitud → <http://localhost:8082/swagger-ui/index.html>
 - Estado_Solicitud → <http://localhost:8081/swagger-ui/index.html>
 - Correccion_De_Datos → <http://localhost:8083/swagger-ui/index.html>
 - Video_De_Consentimiento → <http://localhost:8084/swagger-ui/index.html>

10. OBSERVACIONES FINALES

- Todos los microservicios comparten una misma base de datos MySQL denominada “afores”.
- Los puertos deben configurarse de forma diferente en cada servicio para evitar conflictos.
- Se recomienda verificar la conexión a la base de datos antes de ejecutar los servicios.
- Swagger debe responder correctamente si los controladores y dependencias están activos.

